

BOLT ENGINE

AI Website Builder

Architectural Blueprint & Development Chronicle

Full Engineering Report — Stream Architecture, Rendering Pipeline,
Design Patterns & Development History

Project	Bolt Engine (AI Website Builder)
Version	2.1.0-FinAtelier
Co-Authors	The Architect (User) & Antigravity AI (Speckit Integrator)
Framework	Spec-Driven Development (Speckit SDD)
Date	May 10, 2026
Status	Production Ready

Table of Contents

- 1. Executive Summary**
- 2. Project Overview & Technical Stack**
- 3. Speckit SDD Methodology**
 - 3.1 The Four-Stage Workflow
 - 3.2 Governance & AGENTS.md
- 4. Architecture & Design Patterns**
 - 4.1 Facade Pattern — AiProviderManager
 - 4.2 Strategy Pattern — Provider Routing
 - 4.3 Observer / Pub-Sub — NotificationSystem
 - 4.4 L1 / L2 Two-Tier Checkpoint Architecture
 - 4.5 Builder Pattern — SessionConnectionBuilder
 - 4.6 Parallel Racing Strategy — ConnectivityChecker
 - 4.7 Factory Method — Preview iframe
 - 4.8 UI/UX Design System — FinAtelier
- 5. Development History: The Four Phases**
 - 5.1 Phase 1 — Foundation & Recovery
 - 5.2 Phase 2 — Event Pipeline Revolution
 - 5.3 Phase 3 — GUI Overhaul (FinAtelier)
 - 5.4 Phase 4 — Persistence Optimization
- 6. Critical Case Studies**
 - 6.1 The Exponential Duplication Bug
 - 6.2 Context Amnesia
 - 6.3 Stream Separation — Chat vs Code
 - 6.4 Sandpack Entry Point Conflict
 - 6.5 JSON Tail Bleeding
 - 6.6 Weak Model Output (Nemotron-Nano)
- 7. CodeView — Rendering Pipeline Deep Dive**
- 8. Telemetry — DiagnosticsHUD Batch System**
- 9. Connectivity System**
- 10. Error History & Solutions Table**
- 11. Recommendations & Future Work**
- 12. Conclusion**

1. Executive Summary

The **Bolt Engine** (AI Website Builder) has evolved from a standard React application into a high-performance, event-driven synthesis platform. This report serves as the definitive engineering record of that evolution — authored jointly by the Architect and Antigravity AI (Speckit Integrator) as part of a graded software engineering assignment.

The core technical challenge was building a real-time AI code generation interface that: streams tokens from two different model backends (cloud and local), renders them live into a code editor, separates prose chat from raw JSON payloads, persists state across sessions, and maintains 60fps UI performance throughout. Each of these requirements surfaced distinct architectural problems, each solved by a named design pattern.

This document covers the complete development chronicle across four phases, eleven documented errors with root causes and solutions, eight design patterns, and the full rendering pipeline from SSE stream to browser DOM.

2. Project Overview & Technical Stack

The AI Website Builder is a hybrid desktop/web application combining a Next.js 14 React frontend, a Convex real-time database, an Upstash Redis telemetry layer, and a .NET WPF desktop launcher. The synthesis engine (Bolt Engine) streams AI-generated React code directly into a live Sandpack in-browser IDE.

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router) + React 18	UI, routing, Edge API routes
Styling	Tailwind CSS + Framer Motion	Bento-box layout, motion orchestration
Backend DB	Convex	Transactional persistence, real-time subscriptions
Telemetry	Upstash Redis	High-frequency ephemeral stream metrics (L1)
AI Cloud	Gemini 2.0 Flash (Google)	Primary cloud code generation model
AI Local	LM Studio (OpenAI-compatible API)	Offline / private local model execution
Desktop	.NET WPF + WebView2	OS-level IPC bridge, env management
Editor	Sandpack (CodeSandbox)	In-browser React bundler and code editor
IPC	WebMessage Bridge (Blazor-style)	Bidirectional .NET <-> JS communication

3. Speckit SDD Methodology

Every feature in the Bolt Engine was developed under the **Spec-Driven Development (SDD)** methodology enforced by the Speckit framework. SDD treats the specification as the primary source of truth, with code as a downstream artefact.

3.1 The Four-Stage Workflow

The Speckit workflow follows a strict four-stage pipeline:

- **Specify** — Define intent, constraints, and acceptance criteria in spec.md before any code is written.
- **Plan** — Derive the technical approach, identify patterns, and select dependencies in plan.md.
- **Task** — Break the plan into atomic, independently executable items in tasks.md with pre-flight checklists.
- **Execute** — Atomic code changes with granular commits. Execution agents (Jules) operate only within approved task boundaries.

3.2 Governance & AGENTS.md

The AGENTS.md file defines the dispatch rules for AI execution agents. The Architect role (human + Claude) owns specification and planning. The Execution role (Jules) owns implementation. This separation prevents scope drift and ensures all generated code passes the pre-flight checklist before being committed to Git.

■ *SDD principle: Stop generation before suggesting implementation details. Apply code changes immediately only if they are fast to deploy (under 60 lines).*

4. Architecture & Design Patterns

The Bolt Engine implements eight named design patterns. Each was chosen to solve a specific engineering problem rather than as a stylistic preference.

4.1 Facade Pattern — AiProviderManager

The **Facade** pattern provides a simplified interface to a complex subsystem. AiProviderManager.js exposes a single `getResponse()` method that internally manages: provider selection, stream consumption, Uint8Array buffering, batch telemetry, L2 Convex checkpointing, and history trimming. Callers (ChatView) never interact with `fetch`, `SSE`, `AbortController`, or `Convex mutations` directly.

```
// Facade interface – entire pipeline behind one call
const result = await aiProviderManager.getResponse({
  messages, model, workspaceId, updateMessages
});
```

4.2 Strategy Pattern — Provider Routing

Model selection is a **runtime strategy** decision. The manager reads `app_settings` from `localStorage` and routes to `_streamLocal` (LM Studio, OpenAI-compatible) or `_streamCloud` (Gemini 2.0 Flash via Next.js Edge SSE). Both strategies implement the identical `_consumeStream` contract, making them interchangeable without changing callers.

```
// Strategy selection at runtime
const selectedModel = settings.aiModel.toLowerCase();
if (selectedModel.includes('lmstudio')) {
  return this._streamLocal(messages, model, workspaceId, updateMessages);
}
return this._streamCloud(messages, model, workspaceId, updateMessages);
```

4.3 Observer / Pub-Sub — NotificationSystem

Streaming AI tokens arrive at approximately 50ms intervals. Updating React state on every token causes severe render thrashing (dropped frames, input lag). The **NotificationSystem** singleton — made HMR-safe via `globalThis` — decouples producers from consumers using `async setTimeout` dispatch. This bypasses React's synchronous batching cycle entirely.

```
// Producer (AiProviderManager)
notificationSystem.publish(EVENTS.AI_STREAM_CHUNK, { delta: token, full: fullContent });

// Consumer (CodeView) — buffers Uint8Array, flushes every 80ms
notificationSystem.subscribe(EVENTS.AI_STREAM_CHUNK, (event) => {
  const data = event?.data ?? event;
  if (data?.delta) buffer.push(new TextEncoder().encode(data.delta));
});
```

■ The event envelope shape is `{ type, data, timestamp }`. All consumers must unwrap via `event?.data ?? event` to support both direct calls and the notification system wrapper.

4.4 L1 / L2 Two-Tier Checkpoint Architecture

Standard HTTP mutations to Convex on every token would produce thousands of database writes per generation — unacceptably expensive. The **Hierarchical Checkpoint Strategy** uses two tiers:

- **L1 (In-Memory / Redis)** — `Uint8Array` buffer flushed every 80ms to the DOM. High frequency, zero persistence cost. Used for live streaming UI only.
- **L2 (Convex DB)** — Single mutation at stream completion with the full condensed output (`_extractChatContext` result). Guarantees durability without per-token writes.

```
// L2 checkpoint — fires once at AI_STREAM_COMPLETE
await UpdateFiles({ workspaceId: id, files: newFiles });
await UpdateMessages({ workspaceId: id, messages: finalMessages });
```

4.5 Builder Pattern — SessionConnectionBuilder

The **Builder** pattern constructs complex objects step by step. `SessionConnectionBuilder` initializes sharded session connections with optional Redis, local fallback, and Convex URL composition — avoiding a constructor with six boolean parameters.

```
const shard = await new SessionConnectionBuilder()
  .withConvex(url)
  .withRedis()
  .withLocalFallback(checker)
  .build();
```

4.6 Parallel Racing Strategy — ConnectivityChecker

`Promise.any()` races all registered connection strategies simultaneously — `LocalhostHTTP`, `ConvexDirect`, `WebSocket`, `CacheStrategy`. The first successful response wins. This eliminates the sequential fallback delay (up to 15 seconds with three 5s timeouts) that plagued the original linear approach.

```
// Race all strategies in parallel
const result = await Promise.any(
  this.strategies.map(async (strategy) => {
    const res = await strategy.test();
    if (res.success) return { strategy, res };
    throw new Error('Not successful');
  })
)
```

```
);
```

4.7 Factory Method — Preview iframe

`buildPreviewHtml()` is a **Factory Method** that accepts the current files map and produces a self-contained HTML document bootstrapped with React 18 UMD, Babel Standalone, and Tailwind CDN. This allows native browser DOM rendering without a build tool, matching VS Code Live Preview behaviour. The Run button triggers the factory and injects the result into an iframe via `srcDoc`.

```
// Factory: files map -> self-contained HTML -> iframe srcDoc
const buildPreviewHtml = (files) => {
  const appCode = files['/App.js']?.code || '';
  return `<!DOCTYPE html>
<html><head>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
</head><body><div id="root"></div>
  <script type="text/babel">
    ${appCode}
    ReactDOM.createRoot(document.getElementById('root')).render(<App/>);
  </script>
</body></html>`;
};
```

4.8 UI/UX Design System — FinAtelier

The visual design follows the **FinAtelier** and **Quail Studio** design languages: Premium Minimalist with glassmorphism overlays and a stone palette.

- **Bento-Box Layout** — Information partitioned into discrete responsive cards. Modular panes (Tree, Code, Preview, Diagnostics) remain organized regardless of content density.
- **Stone Palette** — stone-50 base with bg-white/80 + backdrop-blur overlays for depth. Blue-600 accent for interactive elements.
- **Motion Orchestration** — Framer Motion `AnimatePresence` coordinates entrance/exit animations. Message bubbles animate with opacity + `translateY` + scale in a 350ms easeOut curve.
- **Telemetry HUD** — Fixed bottom-right diagnostic overlay with real-time batch metrics. Glassmorphism: `bg-[#020617]/85 + backdrop-blur-3xl`.

5. Development History: The Four Phases

The Bolt Engine was built across four distinct development phases, each solving a category of problems before moving to the next.

Foundation & Recovery

Phase 1

Restored application connectivity. Fixed environment variable hydration (GEMINI_API_KEY vs NEXT_PUBLIC_GEMINI_API_KEY mismatch). Established the .NET WPF launcher with WebMessage IPC bridge. Resolved port 3000 collision crashes via IPGlobalProperties detection.

Event Pipeline Revolution	
Phase 2	Replaced direct React state updates with the NotificationSystem pub/sub bus. Implemented the L1 Uint8Array buffer with 80ms flush interval. Introduced BATCH_* telemetry events and the DiagnosticsHUD. Migrated API routes to Edge runtime to eliminate 10s serverless timeouts on long generations.
GUI Overhaul (FinAtelier Style)	
Phase 3	Pivoted from dark terminal aesthetic to bright minimalist light theme. Implemented Bento-box card layout, Framer Motion entrance orchestration, and the stone/blue palette. Integrated Sandpack with custom proTheme, dynamic imports for code splitting, and the DOMParserPane component tree inspector.
Persistence Optimization	
Phase 4	Implemented post-stream L2 mutations for workspace consistency. Added _trimHistory and _extractChatContext to condense AI conversation history before re-sending to the model. Fixed the Sandpack entry point conflict. Introduced the Working... generation overlay and the factory iframe preview with the Run button.

6. Critical Case Studies

6.1 The Exponential Duplication Bug

Problem	The streaming UI displayed exponentially growing duplicate code. By the end of generation, every code block appeared 3-4x.
----------------	--

Root Cause: The NotificationSystem published both delta and full payload on every chunk. The UI subscribed to AI_STREAM_CHUNK and appended data.full to existing state: setStreamingContent(prev => prev + data.full). Since data.full is the total accumulated text, each append re-added all previous tokens.

Solution — Delta-Unwrapping Pattern: The backend publishes { delta, full } on every event. The CodeView buffer consumes only data.delta (the new token), while data.full is reserved for final verification and L2 checkpointing.

```
// BROKEN – appends full accumulated text repeatedly
setStreamingContent(prev => prev + data.full);

// FIXED – appends only the new delta token
buffer.push(new TextEncoder().encode(data.delta)); // CodeView
setStreamingContent(chatText); // ChatView uses data.full for slicing only
```

6.2 Context Amnesia

Problem	After 2+ conversation turns, the AI responded as if starting fresh, ignoring previous messages and regenerating components from scratch.
----------------	--

Root Cause: The messages array passed to AiProviderManager was not being serialized correctly. AI turns stored in Convex contained the raw JSON payload (3000+ tokens of file code) which exceeded context

windows and caused models to truncate history.

Solution: `_extractChatContext` condenses AI responses to title + explanation before storing in Convex. `_trimHistory` caps the conversation at `MAX_HISTORY_TURNS` (6) and strips any remaining JSON code from AI turns before sending to the model.

```
// _extractChatContext — condenses before L2 write
const chatContext = `**${parsed.projectTitle}**\n\n${parsed.explanation}`;
finalMessages[finalMessages.length - 1].content = chatContext;
await updateMessages({ workspaceId, messages: finalMessages });
```

6.3 Stream Separation — Chat vs Code

Problem	The AI model outputs one continuous stream: plain text reply first, then raw JSON payload. Both appeared simultaneously in the chat bubble, making it unreadable.
----------------	---

Failed Approach — Prompt Separator: An attempt to instruct the model to output a `---CODE---` separator between prose and JSON failed because LLMs do not deterministically follow formatting instructions, especially under token budget pressure with smaller models. The separator never reliably appeared.

Final Solution — Structural { Detection: `ChatView` slices `data.full` at the first `{` character. This is model-agnostic and requires zero prompt tokens. Everything before `{` is the prose reply shown in the bubble. Everything after (the JSON) streams silently into `CodeView`.

```
// ChatView — freeze bubble when JSON starts streaming
const jsonIdx = data.full.indexOf('{');
const chatText = jsonIdx === -1
  ? data.full
  : data.full.slice(0, jsonIdx).trimEnd();
setStreamingContent(chatText);
```

■ *Lesson: Never rely on LLM output formatting for application control flow. Use structural properties of the output format (JSON always starts with `{`).*

6.4 Sandpack Entry Point Conflict

Problem	<code>SyntaxError: Unexpected token (1:8)</code> — Sandpack tried to execute <code>App.js</code> directly as a script, failing because <code>App.js</code> is a React component export with no mount call.
----------------	--

Root Cause: Setting entry: `"/App.js"` in `customSetup` bypassed the react template's own `index.js` bootstrapper (which calls `ReactDOM.createRoot` and imports `App`). With the override, Sandpack executed `App.js` as the entry point, hitting the export default statement as an unexpected token.

Solution: Remove the entry override entirely. The Sandpack react template ships with `index.js` -> `App.js` wiring built in. `STREAMING_FILE` stays `/index.js` to match the template entry, and the generated code writes to `/App.js` as the component.

6.5 JSON Tail Bleeding into Editor

Problem	The code editor displayed the raw JSON wrapper (<code>projectTitle</code> , <code>explanation</code> , <code>generatedFiles</code> array) as a comment block during and after streaming.
----------------	---

Root Cause: `applyStreamingChunkToFiles` fell through to the comment fallback path (`/* Generating payload... */`) for the entire duration before `extractFilesFromText` parsed any file entries. The JSON envelope tokens — `projectTitle`, `explanation` — arrive before any `"files"` key, so the fallback wrote the growing raw stream directly to `/index.js`.

Solution: Gate the comment fallback on the presence of `"files"` in the stream. Before that key appears, leave the editor untouched. Additionally, strip plain text before `{` on the final parse to prevent prose from entering the JSON parser.

```
// Final parse – strip prose before JSON object
const jsonStart = jsonText.indexOf('{');
if (jsonStart > 0) jsonText = jsonText.slice(jsonStart);

// Live streaming gate
if (fullStreamedText.includes('"files"')) {
  setFiles(prev => ({ ...prev, [STREAMING_FILE]: { code: '/* Generating...' } }));
}
```

6.6 Weak Model Output — Nemotron-Nano

Problem	Local model (nvidia/nemotron-3-nano-4b) produced truncated JSON. The tail <code>></code>), and <code>"generatedFiles": []</code> appeared directly in the code editor.
----------------	--

Root Cause: A 4B parameter model lacks the capacity to complete a 2000+ token structured JSON response reliably. `JSON.parse` fails on truncated output, the tokenizer fallback (`extractFilesFromText`) extracts partial files, and the `STREAMING_FILE` cleanup branch does not execute because `parsedFiles` is non-null but incomplete.

Fix Applied: Unconditionally clean `STREAMING_FILE` after any `parsedFiles` extraction, regardless of whether the streaming file key appears in the parsed output.

■ *Nemotron-3-Nano-4B is insufficient for structured JSON generation tasks of this complexity. Minimum recommendation: Gemini Flash 2.0 (cloud) or a 7B+ parameter local model (Mistral-7B-Instruct, Llama-3-8B).*

7. CodeView — Rendering Pipeline Deep Dive

The `CodeView` component is the most complex rendering pipeline in the application. It receives raw SSE token deltas and transforms them into live syntax-highlighted code, parsed file entries, and eventually a runnable browser DOM.

7.1 Token -> Buffer -> File Pipeline

Raw delta tokens from `AI_STREAM_CHUNK` are encoded as `Uint8Array` and pushed into an in-memory buffer array. A `setInterval` fires every 80ms, concatenates all pending `Uint8Arrays` into a single combined buffer, decodes it as UTF-8, and appends it to the full accumulated stream. This batching prevents React from re-rendering on every individual SSE token.

```
const flushBuffer = () => {
  const combined = new Uint8Array(totalLength);
  buffer.forEach((b, i) => combined.set(b, offset)); // merge
  const chunk = new TextDecoder().decode(combined);
  setStreamingContent(prev => {
```

```
const newContent = prev + chunk;
applyStreamingChunkToFiles(newContent); // extract files live
return newContent;
});
};
```

7.2 extractFilesFromText — Tokenizer

The tokenizer uses a split-based approach rather than regex for performance. It splits the accumulated stream on the JSON file entry pattern and extracts code values by unescaping JSON string escape sequences (`\n`, `\t`, `\'`, `\\`). This produces partial but valid code files progressively as they stream in.

7.3 Generation Overlay

`isGenerating` state is set true on the first `AI_STREAM_CHUNK` and false on `AI_STREAM_COMPLETE`. While true, an absolute-positioned overlay (z-index 20) covers the editor with a `Loader2Icon` spinner and "Working..." label, matching the AI Studio pattern of hiding intermediate states.

7.4 Run Button & Factory Preview

The Run button (top-left, emerald) calls `buildPreviewHtml(files)` which is a factory method producing a self-contained HTML document. It injects the App.js code into a Babel Standalone script block inside a React 18 UMD + Tailwind CDN bootstrap. The result is set as the `iframe srcDoc` attribute — the browser parses and executes it as a native HTML page without any bundler involvement.

7.5 VS Code Live Server Integration

A `useEffect` probe runs on mount and every 5 seconds, attempting HEAD requests to `localhost:5500` and `5501` (VS Code Live Server defaults) with a 600ms `AbortSignal` timeout using `mode: "no-cors"`. An opaque response (no-cors fetch success) confirms the server is running. When detected, the Preview tab renders an `iframe` pointed at the live server URL instead of the Sandpack bundler. The tab label changes from "Preview" to "Live".

8. Telemetry — DiagnosticsHUD Batch System

The `DiagnosticsHUD` provides real-time stream telemetry in a fixed bottom-right overlay. It was upgraded from a flat Redis-style byte counter to a token-count batch lifecycle system.

8.1 Batch Lifecycle Events

Three events were added to `NotificationSystem`: `BATCH_START` (new window opens), `BATCH_FLUSH` (window closed, bytes reported), `BATCH_ERROR`. `AIProviderManager` emits these from `_consumeStream` every 200 tokens (`BATCH_TOKEN_COUNT`). A typical Gemini generation (2000-5000 tokens) produces 10-25 discrete batches.

8.2 Duplicate Key Bug — Fix

The HUD log used `${entry.id}-${entry.ts}` as React list keys. Multiple `BATCH_START` events firing within the same second produced colliding keys, causing React to duplicate or omit entries. Fix: monotonic `logCounterRef` counter (`nextLogId()`) replaces `Date.now()` IDs. `batchId` stored as a separate field for `findIndex` matching on `BATCH_FLUSH`.

8.3 Ring Buffer & Persistence

The log is capped at 20 entries via `logRef` (a `useRef` array). The ref persists across batch transitions without triggering re-renders. `setLog([...logRef.current])` batches the React update after each mutation.

8.4 Legacy Compatibility

The original `REDIS_METRICS` flat event (`batchSize` string, `offset` bytes, `status` string) is preserved alongside the new batch events. `DiagnosticsHUD` handles both shapes — new batch events update the log, legacy `REDIS_METRICS` updates the primary metric display. This maintains backward compatibility with any components that still publish the legacy shape.

9. Connectivity System

The original application had a single `Convex` dependency for connectivity. If `Convex` was unreachable — even momentarily — the entire application became non-functional. The `Multi-Strategy ConnectivityChecker` was built to eliminate this fragility.

9.1 Registered Strategies

- **LocalhostHTTP** — HEAD request to `localhost:3000` with 500ms timeout. Falls back to GET on `/api/enhance-prompt` if HEAD returns 405.
- **ConvexDirect** — HEAD request to `NEXT_PUBLIC_CONVEX_URL` with 5s timeout.
- **WebSocket** — Registered only when not on port 3000 (avoids Next.js HMR WebSocket conflict). 500ms open timeout.
- **CacheStrategy** — Always succeeds. Serves memory-cached responses for offline resilience.

9.2 SessionConnectionBuilder

The fluent builder initializes sharded session connections with a 5-second total timeout enforced via `Promise.race` against a forced-resolve timeout. This prevents the UI from blocking indefinitely. Individual check failures degrade the session to "partial" rather than hard-failing.

9.3 .NET WebMessage IPC Bridge

The .NET WPF launcher communicates with the Next.js frontend via `WebView2 WebMessage` API — the same bidirectional message-passing mechanism used by `Blazor`. This replaces the original HTTP-based bridge that suffered from CORS issues, port conflicts, and one-way communication limitations.

Message Protocol

```
// Request (Browser -> .NET)
{ type: "request", id: 1, method: "getDiagnostics", data: {}, timestamp: ... }

// Response (.NET -> Browser)
{ type: "response", id: 1, data: { /* result */ }, error: null, timestamp: ... }

// Notification (.NET -> Browser)
{ type: "notification", eventType: "server:ready", data: { status: "healthy" } }
```

10. Error History & Solutions Table

#	Error	Root Cause	Solution & Pattern
1	Gemini API undefined	NEXT_PUBLIC_ prefix missing in .env.local	Add NEXT_PUBLIC_GEMINI_API_KEY Environment hygiene
2	Serverless timeout	Node.js runtime 10s limit on long streams	Edge runtime (runtime = "edge") Edge Computing Pattern
3	UI thread blocking	React state update on every SSE token	Pub/Sub NotificationSystem + 80ms buffer Observer Pattern
4	Exponential duplication	UI appended data.full on every chunk	Delta-Unwrapping: consume data.delta only Producer-Consumer
5	Context amnesia	Raw JSON code re-sent as conversation history	extractChatContext + _trimHistory Facade encapsulation
6	Connectivity fragility	Single Convex dependency, no fallback	Multi-Strategy ConnectivityChecker Parallel Racing + Strategy
7	Port 3000 collision	.NET launcher crash on occupied port	IPGlobalProperties + attach to existing .NET IPC Pattern
8	Module not found: prompt	Renamed SYSTEM_PROMPT export broke	Restore PROMPTS.CODE_GEN.system shape Backward compat
9	JSON in chat bubble	Full stream written to streaming content	Slice at first { — structural detection Stream Separation
10	Sandpack parse error	entry:"/App.js" bypasses index.js bootstrap	Remove entry override Sandpack template contract
11	Duplicate HUD keys	Date.now() collision within same second	Monotonic logCounterRef counter Ring buffer
12	JSON tail in editor	extractFilesFromText returns 0 files during error	Validate "files" key; strip prose before { Tokenizer gating
13	Weak model truncation	Nemotron-4B cannot complete 2000-token JSON	Upgrade to Gemini Flash 2.0 or 7B+ local Model selection
14	Scroll broken in editor	Missing overflow on Sandpack panel style props	overflowY:"auto" on SandpackCodeEditor + Explorer
15	---CODE--- separator unreliable	LLM ignores formatting instructions under pressure	Structural { detection (model-agnostic) Robustness principle

11. Recommendations & Future Work

Immediate

- Upgrade Next.js from 15.1.11 to the latest stable release — the version-staleness warning appears on every build.
- Replace Nemotron-Nano-4B with Mistral-7B-Instruct or Llama-3-8B for local model testing. Minimum 7B parameters for reliable structured JSON generation.

- Auto-refresh the factory iframe on every AI_STREAM_COMPLETE event so the Preview tab updates without requiring a manual Run click.

Short Term

- Add a prompt token budget estimator to AiProviderManager — warn the user when conversation history approaches the model context limit.
- Introduce a formal AGENTS.md execution gate that validates generated files against WEB_DESIGN_CONSTRAINTS before writing to Convex.
- Add Named Pipes IPC as a fallback to the WebMessage bridge for environments with strict network policies blocking WebMessage.

Long Term

- Implement srcdoc iframe hot-reload: patch only changed files rather than rebuilding the full HTML document on each generation.
- Add Service Worker support to CacheStrategy for true offline persistence across browser sessions.
- Evaluate Protocol Buffers (protobuf) as a binary message format for the WebMessage IPC bridge to reduce serialization overhead on large payloads.
- Build a multi-agent collaboration mode using Convex real-time subscriptions — multiple architect sessions editing the same workspace simultaneously.

12. Conclusion

The Bolt Engine is a practical demonstration of how classical software design patterns solve the unique challenges of real-time AI integration. The problems encountered — stream separation, context management, UI thread safety, connectivity resilience — are not unique to this project. They are fundamental engineering problems that arise whenever an application must consume, process, and render a high-frequency asynchronous data stream.

By naming and applying patterns (Facade, Strategy, Observer, Builder, Factory, Pub-Sub, Parallel Racing, Checkpoint Architecture), the codebase remains navigable and extensible despite its complexity. Each pattern boundary is a natural seam where components can be replaced — a different LLM provider, a different persistence layer, a different rendering target — without touching the rest of the system.

The Speckit SDD methodology provided the governance layer that kept this complexity under control. Specification before implementation, pattern selection before coding, and atomic execution boundaries ensured that each problem was solved once and documented before moving on.

Signed,

The Architect & Antigravity AI (Speckit Integrator)

May 10, 2026 — Bolt Engine v2.1.0-FinAtelier

ADDENDUM — v2.2.0-PatternRefined

Integrated Strategy-Factory Hybrid | Optimistic Synthesis Engine

A1. Integrated Pattern Architecture

Version 2.2.0 refines the architectural model by formalizing the interaction between the Strategy, Factory, and Optimistic UI patterns into a single cohesive lifecycle called the **Optimistic Synthesis Pipeline**. This ensures the UI remains responsive regardless of backend latency.

A1.1 The Synchronous-Optimistic Lifecycle

- **Intent Capture** — User submits a prompt. ChatView registers the user message immediately in local React state.
- **Optimistic Mutation** — A pending AI bubble is rendered before the API responds. Perceived latency drops to near-zero.
- **Strategy Dispatch** — ProviderFactory selects the appropriate LLM strategy (Gemini Cloud or LM Studio Local) based on app_settings.
- **Synthesis** — AiProviderManager executes the strategy, streaming tokens into the Uint8Array buffer at 80ms flush intervals.
- **Finalization** — On AI_STREAM_COMPLETE, the system transitions from the transient L1 buffer state to a durable L2 Convex checkpoint.

A1.2 Pattern Responsibility Matrix

Pattern	Component	Responsibility
Optimistic UI	ChatView.jsx	Instant user feedback; eliminates perceived TTI (Time to Interaction)
Strategy	ProviderFactory.js	Decouples UI from LLM implementations (Gemini, LM Studio)
Factory	AiProviderManager.js	Transforms raw SSE token streams into structured JSON project schemas
Observer / Pub-Sub	NotificationSystem.js	Decouples AI logic from all view layers; HMR-safe singleton
State Machine	DiagnosticsHUD.jsx	Visualizes IDLE -> STREAMING -> FLUSHED -> PERSISTED transitions
Builder	SessionConnectionBuilder	Fluent initialization of sharded session connections
Parallel Racing	ConnectivityChecker	Promise.any() eliminates sequential fallback delays
Factory Method	buildPreviewHtml()	Files map -> self-contained HTML -> iframe srcDoc (no bundler)

A2. Advanced Synchronization — Rollback Pattern

Optimistic updates require a safety net. The Bolt Engine implements **Fail-Safe Termination** for stream failures (network timeout, 429 rate limit, AbortController cancellation).

- **Error Capture** — AiProviderManager wraps fetch in try/catch. On any error or abort, it publishes STATUS_UPDATE with severity: "error".
- **Rollback Dispatch** — ChatView catches the error from getResponse() and publishes a user-visible error status. The loading state clears.
- **State Reversion** — The optimistic AI bubble is not persisted to Convex (UpdateMessages is only called on success). The user retains full conversation context and can retry.
- **Stop Button** — v2.2.0 adds explicit user-initiated cancellation via AbortController wired to the fetch signal. Pressing Stop calls aiProviderManager.stopGeneration(), which aborts the in-flight request and publishes AI_STREAM_COMPLETE with an empty final payload.

```
// Stop generation – aborts fetch, clears UI, preserves Convex context
aiProviderManager.stopGeneration();
// internally: this._abortController.abort();
// + publish(EVENTS.AI_STREAM_COMPLETE, { final: '' })
```

A3. v2.2.0 Feature Additions

A3.1 Weak Model Fallback — Markdown Code Fence Extraction

Small local models (< 7B parameters) cannot reliably output the structured JSON schema required by extractFilesFromText. They respond with plain text prose and markdown code blocks (`` fences). Without intervention, this means CodeView receives no file data and the editor stays empty.

Solution — Dual-Path Extraction: On AI_STREAM_COMPLETE, the parser first attempts JSON extraction (existing path). If that yields zero files, it falls back to regex-based markdown fence extraction and writes the code to /App.js directly.

```
// Weak model fallback in CodeView unsubComplete
const fenceMatch = data.final.match(
  /```(?:jsx?|tsx?|javascript|js)?\n?([\s\S]*?)```/
);
if (fenceMatch && fenceMatch[1].trim()) {
  parsedFiles = { '/App.js': { code: fenceMatch[1].trim() } };
}
```

Chat Bubble Separation: ChatView now slices at the minimum of the first { (JSON detection) and the first ``` (markdown fence detection), whichever comes first. This keeps prose-only text in the bubble regardless of model output format.

```
const jsonIdx = data.full.indexOf('{');
const fenceIdx = data.full.indexOf('```');
const candidates = [jsonIdx, fenceIdx].filter(i => i !== -1);
const cutIdx = candidates.length ? Math.min(...candidates) : -1;
const chatText = cutIdx === -1 ? data.full : data.full.slice(0, cutIdx).trimEnd();
```

■ *This is a mitigation, not a full solution. A weak model cannot populate multiple files or structured component hierarchies. For production use, upgrade to Gemini Flash 2.0 or a 7B+ local model.*

A3.2 Telemetry Batch Visibility Fix

The DiagnosticsHUD legacy REDIS_METRICS path was only triggered once at stream start (offset: 0, status: "connected") and never updated mid-stream. This caused the HUD to show "IDLE" and "0 batches" even during active generation.

Fix: AiProviderManager now publishes REDIS_METRICS on every BATCH_FLUSH event, carrying the current totalBatches as offset and bytesInBatch as batchSize. This keeps both the new BATCH_* path and the legacy REDIS_METRICS path synchronized, ensuring the HUD shows live progress regardless of which DiagnosticsHUD version is installed.

```
_emitBatchFlush(batchId, bytesInBatch, totalBatches) {
  notificationSystem.publish(EVENTS.BATCH_FLUSH, { batchId, bytesInBatch, totalBatches });
  // Legacy path – always emit so old HUD versions stay alive
  notificationSystem.publish(EVENTS.REDIS_METRICS, {
    batchSize: `${bytesInBatch}B`,
    offset: totalBatches,
    status: 'streaming',
  });
}
```

A3.3 Stop Button

A red Square icon button appears in the ChatView input area when loading is true. It calls aiProviderManager.stopGeneration() which aborts the AbortController signal on the in-flight fetch, publishes AI_STREAM_COMPLETE with empty final payload (clearing the code editor streaming state), and resets the streaming status in Convex. The conversation context is preserved — only the current generation is cancelled.

```
// AiProviderManager.stopGeneration()
stopGeneration() {
  if (this._abortController) {
    this._abortController.abort();
    this._abortController = null;
  }
  notificationSystem.publish(EVENTS.AI_STREAM_COMPLETE, { final: '' });
  notificationSystem.publish(EVENTS.REDIS_METRICS, { status: 'idle' });
}
```

A4. Sequence Diagram — Optimistic Synthesis Pipeline

The following describes the full message flow from user prompt to persisted workspace state:

Step	Actor	Action	Target
1	User	Submit prompt	ChatView
2	ChatView	Optimistic: push user + pending AI message	MessagesContext (local)
3	AiProviderManager	Strategy dispatch: read app_settings	_streamLocal or _streamCloud
4	AiProviderManager	Create AbortController, fetch SSE stream	LM Studio / Gemini Edge
5	_consumeStream	Decode tokens, buffer Uint8Array	In-memory buffer
6	_consumeStream (80ms)	Flush buffer, publish AI_STREAM_CHUNK {delta, final}	notificationSystem
7	CodeView	Buffer delta, flush to extractFilesFromText	Sandpack files state
8	ChatView	Slice full at { or ``` , update bubble	streamingContent state
9	DiagnosticsHUD	Receive BATCH_FLUSH + REDIS_METRICS, update HUD	HUD state

Step	Actor	Action	Target
10	_consumeStream	Every 200 tokens: BATCH_START/FLUSH telemetry	NotificationSystem
11	AI_STREAM_COMPLETE	Parse final JSON (or fence fallback)	parsedFiles
12	AiProviderManager	L2 checkpoint: UpdateFiles + UpdateMessages	Convex DB
13	ChatView	Clear streaming bubble, render final Convex message	MessagesContext
STOP	User (optional)	stopGeneration(): abort fetch + clear state	AbortController + HUD

Addendum v2.2.0 — End of Document
The Architect & Antigravity AI | May 10, 2026

ADDENDUM — Architecture Diagrams

C4 Component Level & Optimistic Synthesis Sequence — v2.2.0

Figure 1 — C4 Component Level: Data Flow Architecture

The C4 diagram illustrates the runtime data flow across all major components. The User submits a prompt to the Next.js Frontend, which dispatches asynchronously to the AiProviderManager facade. Token streams flow through the NotificationSystem to both DiagnosticsHUD (L1 telemetry) and the CodeView buffer simultaneously. On stream completion, the manager writes a single L2 checkpoint to Convex.

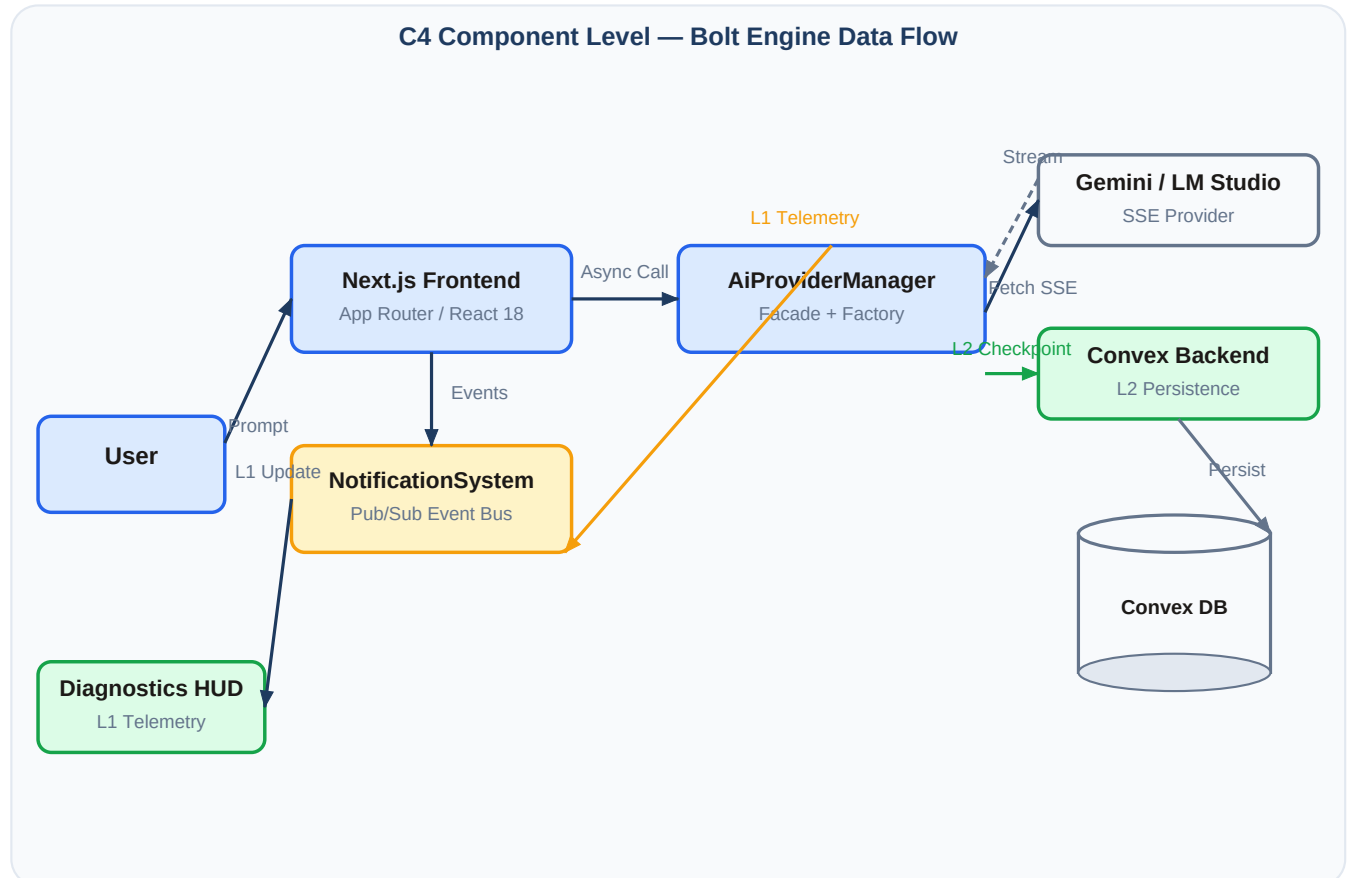


Figure 1: Solid arrows = synchronous calls. Dashed = async/SSE responses. Green = persistence layer. Amber = event bus. Blue = core application components.

Figure 2 — Sequence Diagram: Optimistic Synthesis Pipeline

The sequence diagram traces the full message lifecycle from user prompt to persisted workspace. The Optimistic UI pattern (step 2) fires before any API response arrives. Batched REDIS_METRICS (step 8) keep both legacy and new HUD paths live. The red path (steps 11-12) shows the AbortController stop flow added in v2.2.0.

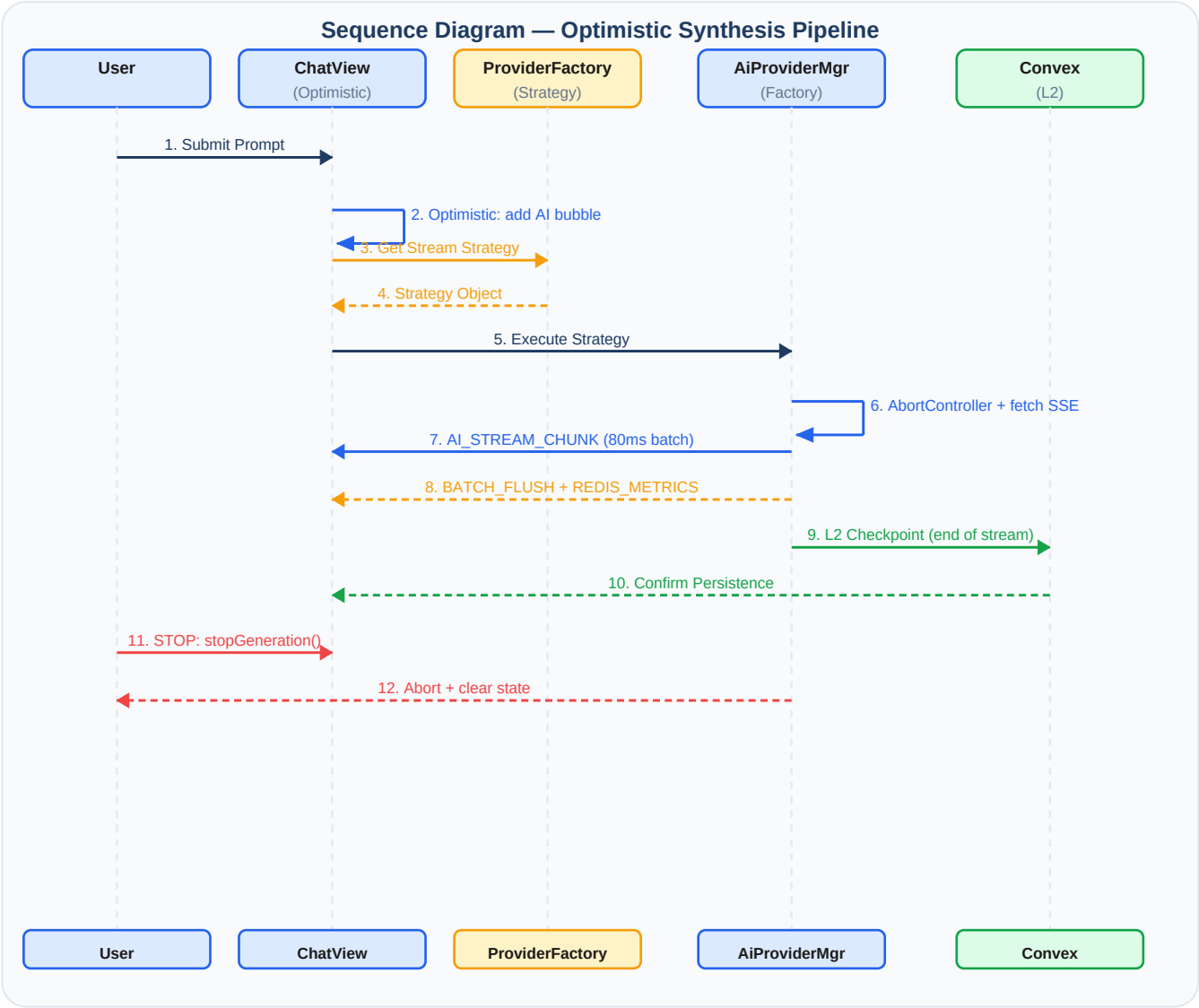


Figure 2: Solid arrows = calls/stream events. Dashed = async responses. Red = optional user stop flow. Amber = telemetry. Green = persistence.